Repository validation summary

Review date: 2026-10-29. This record supersedes the July 2026 snapshot and describes source validation, not deployed Azure state.

Scope and method

Reviewed the root Terraform composition, Dev/Sandbox/Prod inputs, GitHub Actions, Azure DevOps YAML/templates, repository scripts, documentation portal, and checked-in demo/reference content. Existing local edits and four already-created commits were preserved.

Terraform validation used a clean local temporary workspace and the upstream repository checked out at `2e0d476c1e80c327162ff59147bff16454e4af3d`. To avoid repeatedly downloading the same full repository for each module on the shared drive, validation substituted equivalent local paths to that exact pinned checkout. Root provider selections and checksums came from the committed lockfile. The network Git module downloader was not independently exercised for every root module.

Results

Check	Result and scope
Terraform format	Passed `terraform fmt -check -recursive -diff` with Terraform 1.15.8 for root and environment source.
Terraform init / validate	Backend-free initialization with a read-only lockfile passed; root validation passed with zero errors and zero warnings against the pinned modules. Provider checksums were verified for Windows and Linux AMD64. No Azure backend or live plan/apply was run.
TFLint	No errors with the default Terraform ruleset. 27 unused-input warnings remain for compatibility inputs retained by the root API. Error-severity validation succeeds; warning output is retained for future interface cleanup.
Trivy root configuration	No HIGH/CRITICAL findings in the scoped authored composition scan, with the separate upstream source tree excluded. This is not a dependency-wide clean bill of health.
Trivy pinned upstream modules	Four HIGH `AZU-0039` findings in `modules/fortigate/main.tf`, where appliance VM resources permit password authentication. No CRITICAL findings in this scan. Upstream source was reviewed without maintaining a local fork.
GitHub workflow lint	Actionlint 1.7.12 passed; the local check disabled external ShellCheck/Pyflakes integrations and used separate Bash syntax checks.
YAML / script syntax	Passed workflow, Azure DevOps, template, and pre-commit YAML parsing; PowerShell parser checks; JavaScript syntax; and Bash syntax for 12 scripts.
Content and documentation	Passed relative-link validation for 44 Markdown files, all 20 demo HTML files, the six-site reference suite validator, and generation/link checks for 39 documentation pages. Four refreshed PDFs were rendered and visually reviewed.
Security regressions	Passed URL/output boundary and Key Vault review-mode checks; nine isolated Terraform input tests passed. Invalid installer versions and corrupt archives were rejected before extraction/install. Process-scoped Git authentication worked with a synthetic token.
Secrets	Trivy filesystem secret scanning of the staged-source snapshot passed with zero findings. The scan does not inspect Git history or external secret-store contents.

Changes recorded

- Immutable module/action references, bounded provider compatibility, provider lockfile tracking, and a tested CLI version.

- Explicit workload ingress and shared data-plane allowlists; shared storage and Key Vault deny-by-default ACLs; key-based Linux SSH defaults and no direct root SSH.

- Credential-free PR/static checks, authenticated plans restricted to trusted events, static checks required before planning, one-day GitHub plan retention, and removal of obsolete module-download PAT requirements.

- Verified Terraform archive downloads, strict formatting, process-scoped module credentials, and runner cleanup logging that never prints header values.

- Explicit Key Vault copy targets, review mode, same-vault rejection, and nonzero failure for partial copies.

- Safe documentation cleanup boundaries, URL scheme filtering, real link checks, normalized source formatting, current module documentation links, and refreshed generated references.

See CHANGELOG.md (../../CHANGELOG.md) for the dated work record and repository security controls (../60-security-governance/repository-security.md) for operational details.

Remaining limits and rollout requirements

1. Verify administrator SSH keys, recovery access, and runner egress before applying. Shared storage/Key Vault public access now requires an approved address; their private endpoints remain disabled. GitHub-hosted runner addresses are not automatically trusted.

2. The existing environment example address was carried over from the prior workload NSG; it has not been certified as current runner egress. Review it before deployment.

3. Resolve or accept the FortiGate appliance password-authentication findings in the upstream module release process. Constrain administrative source CIDRs and protect appliance credentials meanwhile.

4. Review retained compatibility inputs before removing them in a separate interface change. No TFLint rule was globally disabled to hide the warnings.

5. Demo CSP still allows inline scripts/styles and external HTTPS images. Strict CSP requires a further content refactor.

6. Environment approvals, OIDC federation, Azure RBAC, live service reachability, dependency CVE coverage, and existing Git history were not verified by these source checks.

7. Production pricing/reference links and external service behavior were not exhaustively revalidated. Documentation source links and local references were checked; archived external module reports retain their original scope.